# Network Science

## Lecture 04: Communities and Graph Partitioning

Prof. Dr. Michael Granitzer

Dr. Kanishka Ghosh Dastidar

Dr. Jelena Mitrovic

# From Classifying Edges to Classifying Nodes and Groups

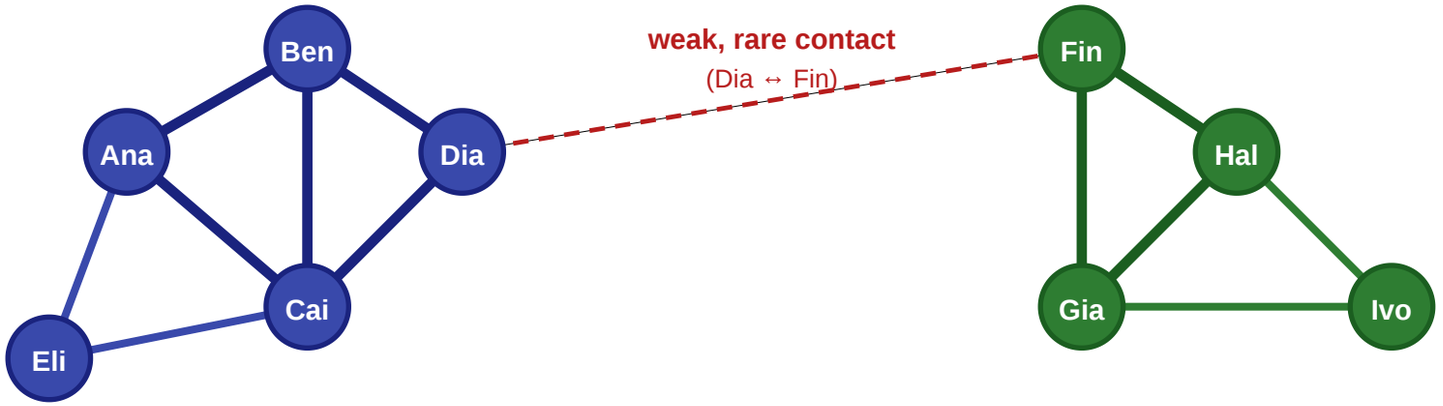
In Lecture 03 we asked which *edges* carried novel information — strong vs. weak, bridges vs. embedded. Today we ask which *nodes* occupy important positions, and how the graph splits into dense groups.

Let us return to the same workplace.

# The Same Scenario, New Questions

**A small workplace: five questions in one picture**

nodes = employees · thick edges = close contacts · thin edges = acquaintances




**Questions we'd like to answer from this picture**

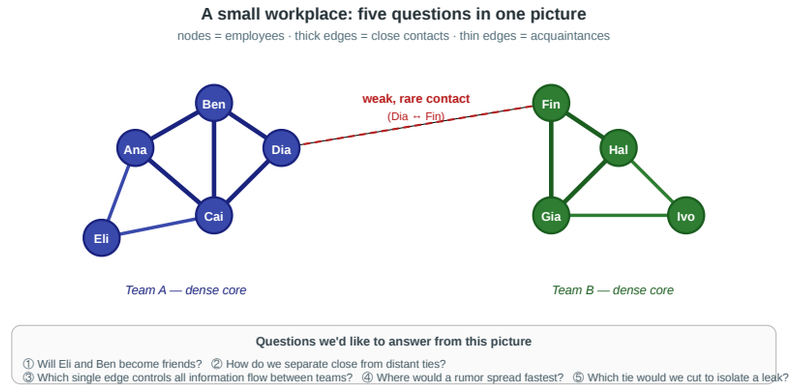
① Will Eli and Ben become friends? ② How do we separate close from distant ties?
③ Which single edge controls all information flow between teams? ④ Where would a rumor spread fastest? ⑤ Which tie would we cut to isolate a leak?

Same graph as last time — two teams linked by a single weak Dia–Fin contact. Lecture 03 asked about the edges. Today the questions are about the nodes and the groups.

# Four Group-Level Questions

1. **Importance.** Who is the single most important person in this office — and what does "important" even mean?
2. **Boundaries.** Where is the line between team A and team B, and could an algorithm discover it *without being told the labels*?
3. **Uniqueness.** If we ran community detection on a 10 000-person company, would the answer be unique — or would different methods give different maps?
4. **Validation.** How do we know when a detected community is real rather than an artifact of the algorithm?

Every concept in today's lecture is a formal answer to one of these questions.

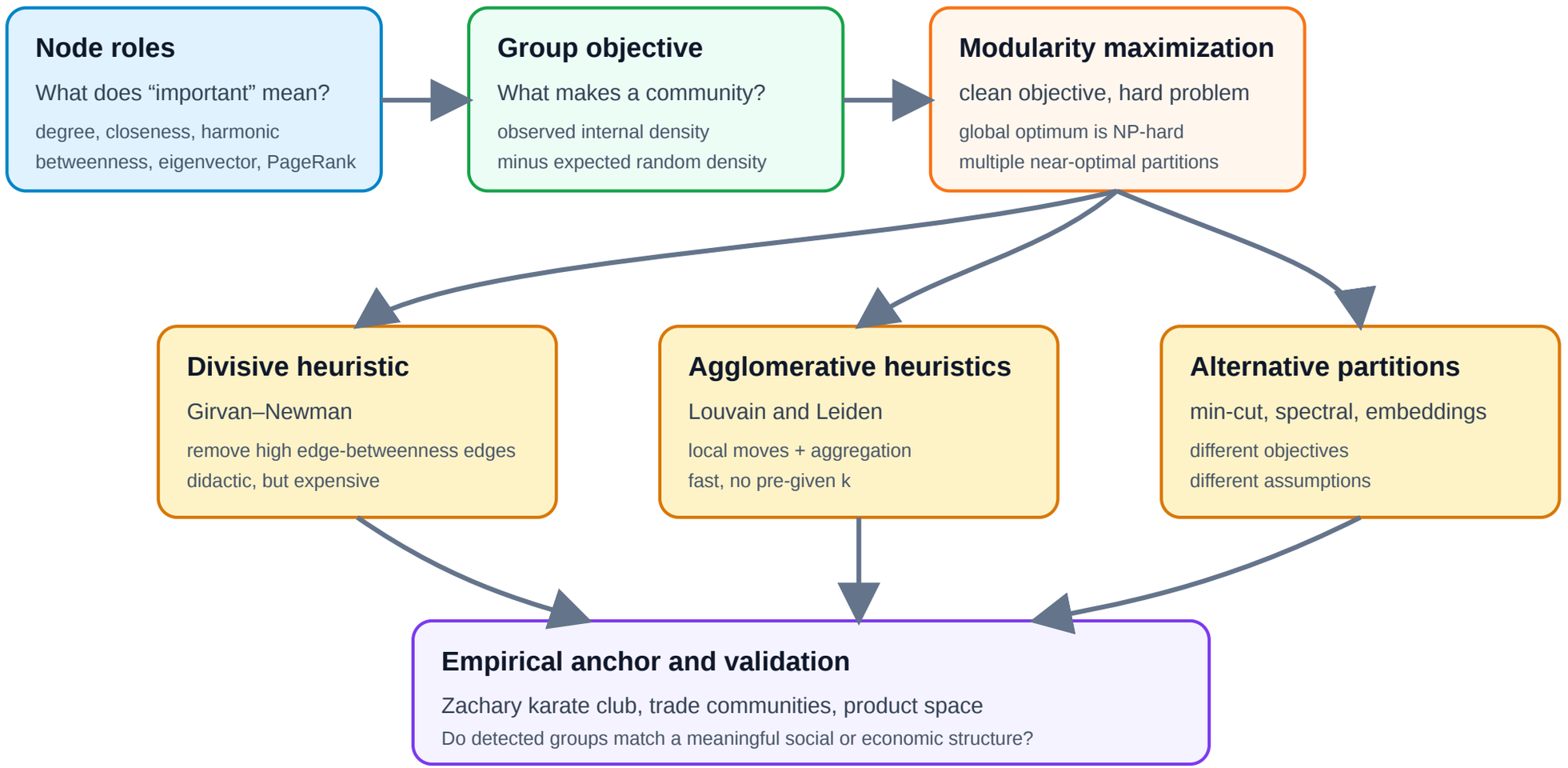


# Theory vs. Measurement — Continued

The gap we opened in Lecture 03 does not close in Lecture 04. Every concept today has a theoretical form we cannot compute on real data and a measurable proxy we can.

| Question | Theoretical construct | Measurable proxy |
|---|---|---|
| Who is most important? | *(no single answer — multiple theories)* | Degree / closeness / betweenness / eigenvector |
| What makes a group a group? | Community with high internal density | Modularity $Q$ |
| Best partition? | Global modularity maximum (NP-hard) | Girvan–Newman / Louvain heuristics |
| Is a partition meaningful? | Agreement with ground-truth labels | Benchmark datasets (Zachary's karate club) |

# Argument Map



# Takeaway

L03 classified edges; L04 classifies nodes and groups. The modeling–measurement loop is the same: a theoretically clean objective is NP-hard, polynomial-time heuristics approximate it, and empirical benchmarks test whether the heuristics recover meaningful structure.

# Motivation: Roles and Social Capital in a Social Network

**Guiding Question:** What kinds of structurally different positions can people occupy inside the same network?

# Three Structural Positions

**Embeddedness.** A node whose neighbors are themselves densely interconnected. High clustering coefficient.

**Brokerage.** A node connecting otherwise separated groups. Low clustering coefficient, high betweenness.

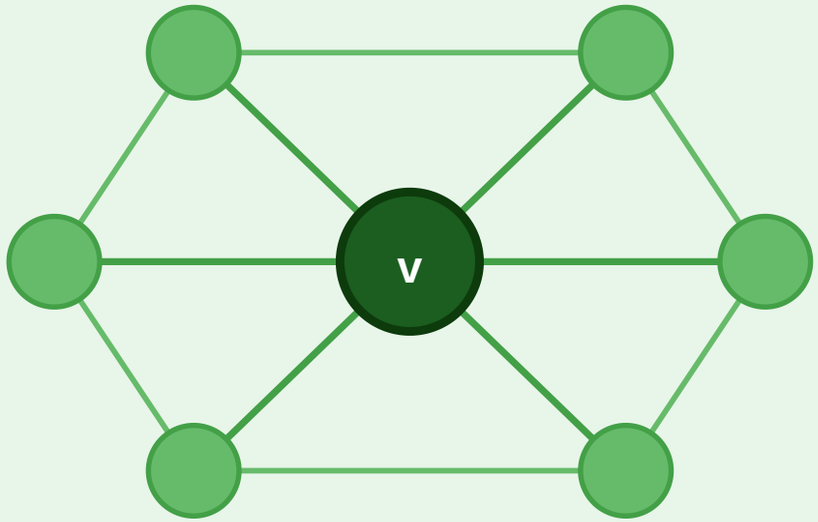
**Structural hole.** The empty region in the network that a brokerage position spans.

They are related but not identical: **the hole is the missing connection between groups; the broker is the actor positioned across that missing connection.**

# Embedded vs. Broker

## Embedded Node

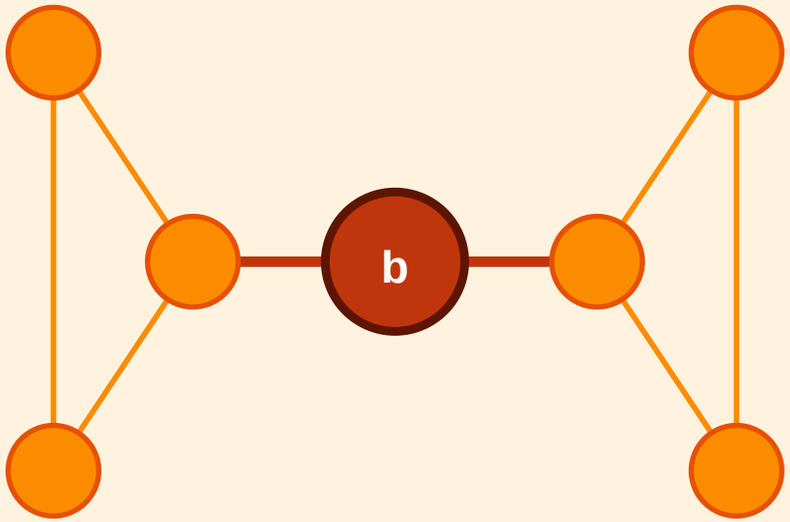
Dense, mutually connected neighbourhood




High clustering · trust · redundant info

## Broker (Structural Hole)

Sole link between two separated clusters

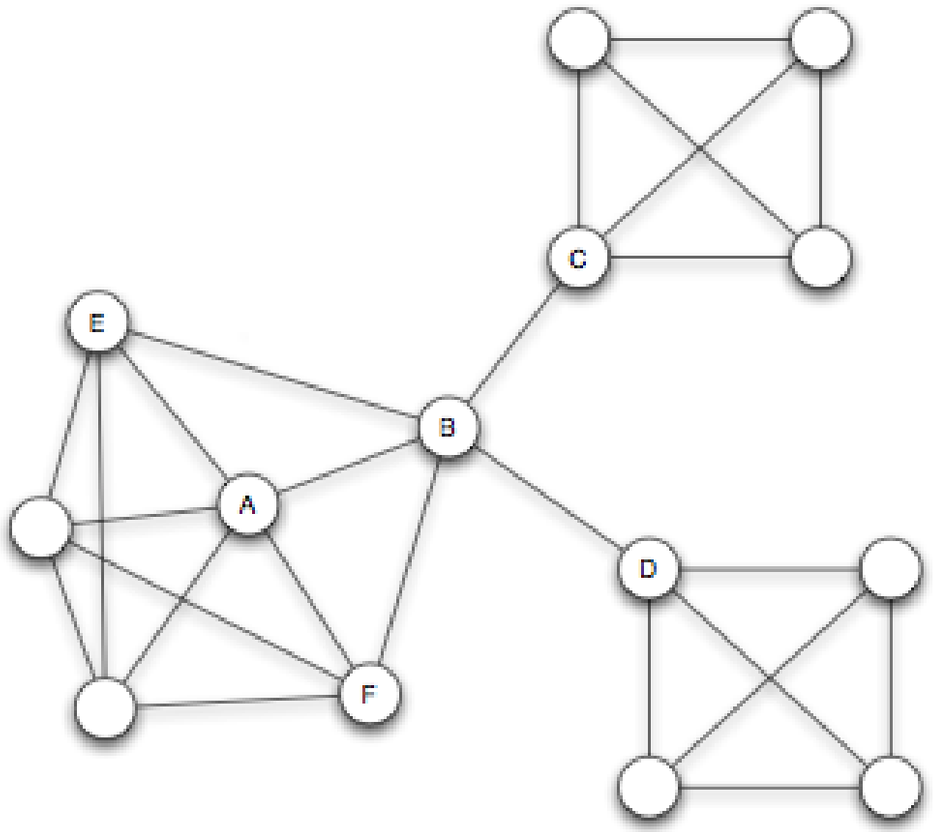



Low clustering · high betweenness · novel info

The embedded node $v$ sits inside a tightly connected neighborhood — its removal barely affects connectivity. The broker $b$ is the only short link between two clusters — its removal would isolate them.

# Structural Holes as Social Capital

A node that acts as the sole short link between otherwise separated groups spans a *structural hole* (Burt, 1992).




*Source: Burt 1992*

- **Information advantage:** the broker receives early, non-redundant signals from independent groups.
- **Control advantage:** the broker mediates flows between groups that cannot communicate directly.

**Not the same thing:** remove the missing gap and the structural hole disappears; remove the spanning actor and the brokerage position disappears.

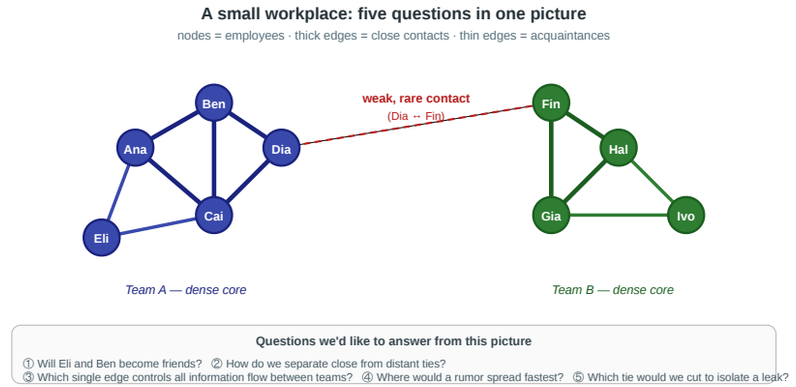
# Takeaway

> "Importance" is not one thing. Brokerage, embeddedness, and degree capture different social advantages — and the same node can sit in multiple roles depending on the question.

# Quick Check

Look again at the workplace figure.

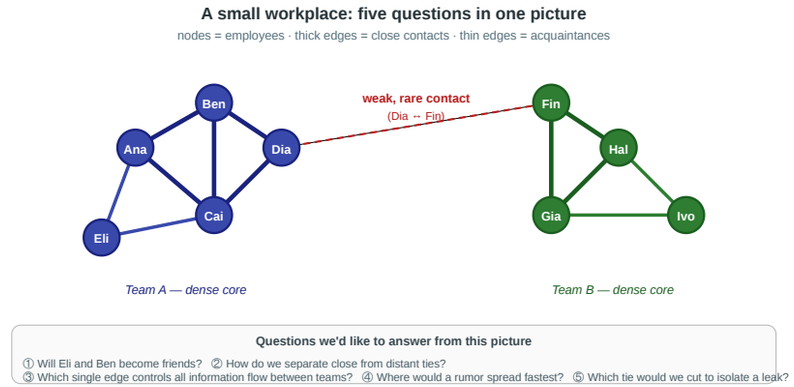
1. Which of these nine people is the clearest *broker*, and what structural feature makes them one?
2. Which is the most clearly *embedded*?
3. What would change structurally if Dia left the company tomorrow?



# Quick Check

Look again at the workplace figure.

1. Which of these nine people is the clearest *broker*, and what structural feature makes them one?
2. Which is the most clearly *embedded*?
3. What would change structurally if Dia left the company tomorrow?




**Solution:** Dia is the broker because the cross-team edge runs through her. Ana, Ben, and Cai are embedded in the left triangle; Ben is strongest because his neighbors are mutually connected. If Dia leaves, the Dia–Fin edge disappears and the two teams become disconnected components.

# Centrality

**Guiding Question:** How should we measure which nodes are important?

# Centrality

**Guiding Question:** How should we measure which nodes are important?

$$\text{importance} \neq \text{degree alone}$$

# Degree Centrality — Local Exposure

$$C_D(v) = \frac{\deg(v)}{n-1}$$

**Raw form.** $C_D^{raw}(v) = \deg(v)$

**Meaning.** How many direct contacts does $v$ have?

**Theory.** Direct exposure: importance comes from immediate contacts.

**Range.** Raw: $0$ to $n-1$; normalized: $0$ to $1$.

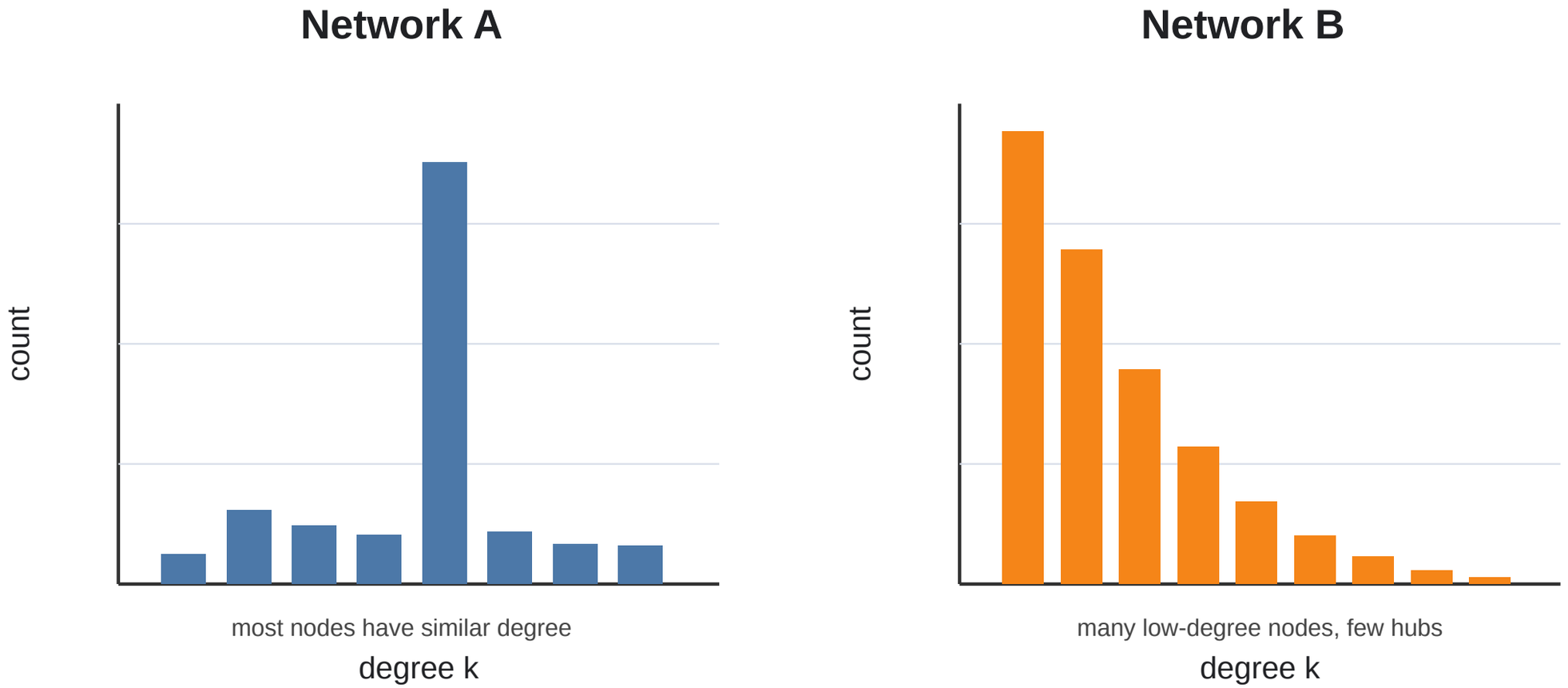
**Complexity.** All scores: $O(n+m)$ including initialization; sorted ranking adds $O(n \log n)$.

**Example.** In a village gossip network, high-degree people hear from many neighbors directly.

**Network diagnostic.** The **degree distribution** is a quick property check: equal-sized degrees suggest homogeneous structure; a heavy-tailed / power-law-like distribution suggests hubs and inequality (Barab'asi & Albert 1999).

# Use of Degree Centrality — Degree Distribution

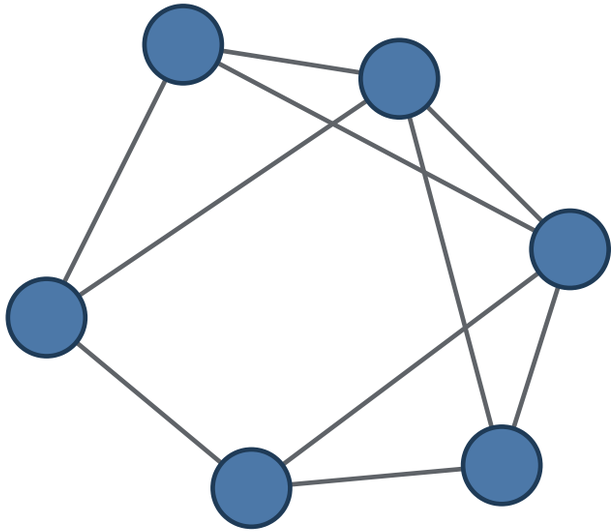
Show the degree distribution either as (i) histogram (below) or (ii) sorted by degree count to highlight distributional properties.




Make a guess: **what could a network with this degree distribution look like?**

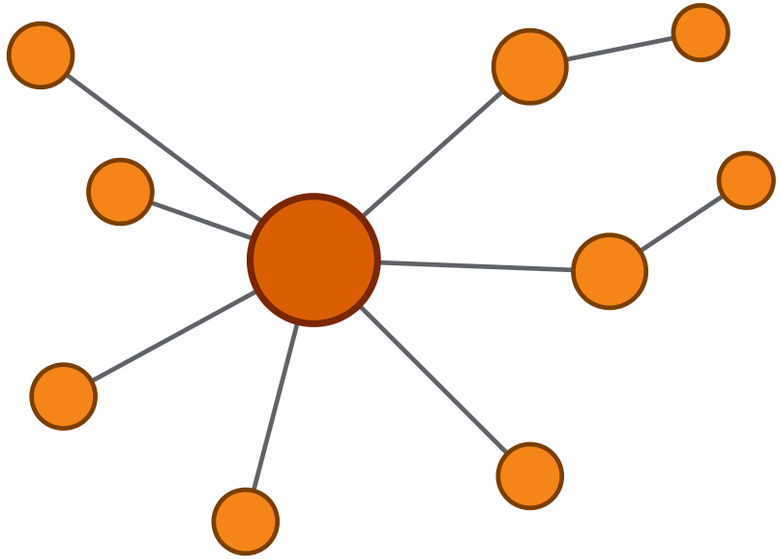
# Degree Distribution — Possible Graphs

## A: homogeneous



similar degree, no obvious hub

## B: hub-dominated



few hubs, many peripheral nodes

These are not unique reconstructions. Many graphs can share the same degree distribution, but the distribution gives a useful first guess about whether the network is homogeneous or hub-dominated.

# Closeness Centrality — Short-Path Access

$$C_C(v) = \frac{n-1}{\sum_{u \neq v} d(v,u)}$$

**Distance input.** Shortest-path distances $d(v,u)$ from $v$ to all other nodes.

**Meaning.** How near is $v$ to all other nodes on average?

**Theory.** Accessibility: importance comes from short paths to everyone.

**Range.** Normalized: $0$ to $1$; $1$ only if $v$ is adjacent to everyone.

**Complexity.** One node: $O(n+m)$ BFS; all nodes: $O(n(n+m))$ unweighted.

**Example.** An emergency-response hub with low average travel distance to all districts has high closeness.

**Use.** Closeness appears in accessibility studies: which hospital, station, or facility minimizes network distance to a population center

# Harmonic Centrality — Robust Reachability

$$H(v) = \sum_{u \neq v} \frac{1}{d(v,u)} \quad \text{with } \frac{1}{\infty} = 0$$

**Reciprocal-distance idea.** Replace distances by $1/d(v,u)$ before summing.

**Meaning.** Many reachable nodes through short paths.

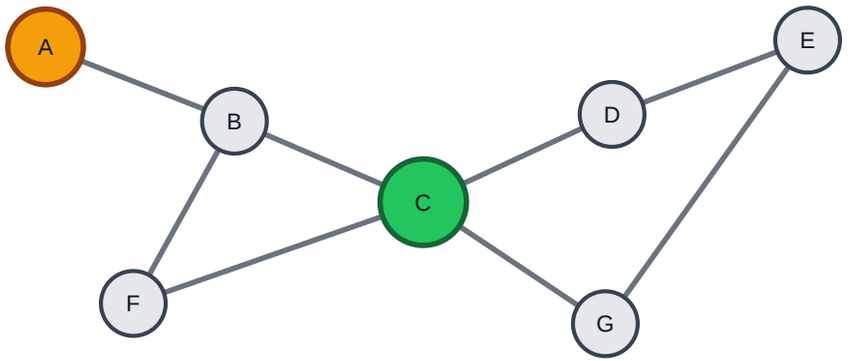
**Theory.** Robust accessibility: unreachable nodes contribute $0$, not $\infty$.

**Range.** $0$ to $n-1$; $n-1$ means direct access to everyone.

**Complexity.** One node: $O(n+m)$ BFS; all nodes: $O(n(n+m))$ unweighted.

Harmonic centrality is the disconnected-graph extension of closeness: unreachable nodes contribute $0$ instead of making the distance sum ill-defined.

# Use of Closeness Centrality — Accessibility




C is a better service hub: short paths to almost every district.
A is connected, but peripheral: average distance is larger.

**Question.** Where should we place a service hub so it can reach everyone quickly?

**Workflow.**

1. Model roads, rail, or communication links as weighted edges.
2. Compute closeness or harmonic closeness.
3. Compare candidate locations against demand or population.

**Class prompt:** ask which node is the best emergency hub, then ask what changes if one district becomes disconnected.

# Betweenness Centrality — Brokerage and Control

$$C_B(v) = \sum_{s \neq v \neq t} \frac{\sigma_{st}(v)}{\sigma_{st}}$$

$\sigma_{st}$ is the number of shortest $s$-$t$ paths; $\sigma_{st}(v)$ counts those passing through $v$.

**Meaning.** How often does $v$ sit between other nodes?

**Theory.** Brokerage: importance comes from sitting on paths between others.

**Range.** Normalized: $0$ to $1$; $1$ means all shortest paths between others use $v$.

**Complexity.** Exact Brandes: $O(n(n+m))$ unweighted; weighted graphs are slower.

**Difference from closeness.** Closeness asks: "How near is $v$ to everyone?" Betweenness asks: "How many shortest paths between *other* nodes depend on $v$?"

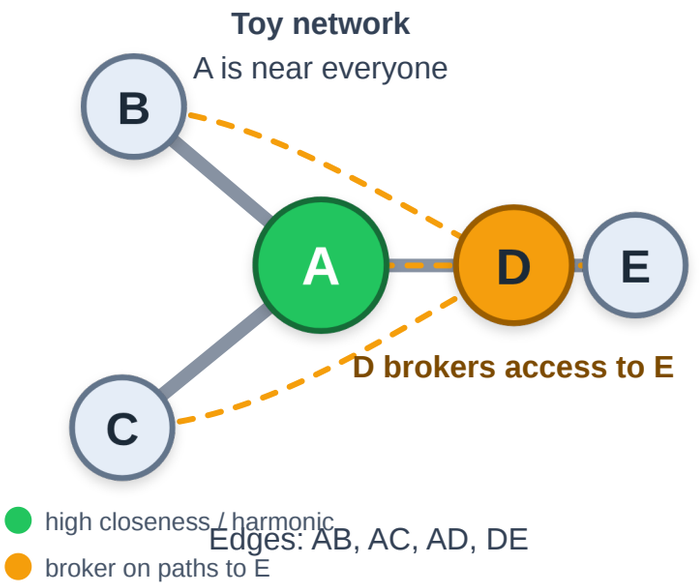
Exact betweenness for one focal node is not just one BFS: it still needs shortest-path dependency information across many source-target pairs. Brandes computes all nodes for essentially the same asymptotic cost.

# Computing Closeness, Harmonic, and Betweenness

Toy network: $A$ is connected to $B, C, D$; $D$ is connected to $E$. Here $n = 5$ and normalized betweenness divides by $\binom{4}{2} = 6$ possible pairs among the other nodes.

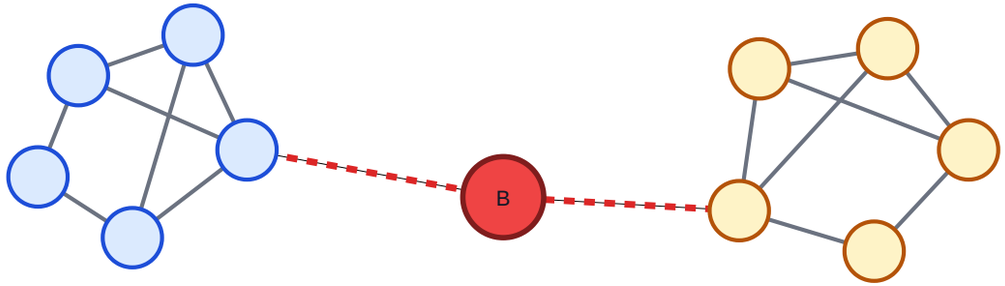
| Node | Distances | Closeness | Harmonic $H(v) = \sum 1/d$ | Harmonic mean distance |
|---|---|---|---|---|
| $A$ | $1, 1, 1, 2$ | $4/5 = 0.80$ | $3.50$ | $4/3.50 = 1.14$ |
| $D$ | $1, 1, 2, 2$ | $4/6 = 0.67$ | $3.00$ | $4/3.00 = 1.33$ |
| $E$ | $1, 2, 3, 3$ | $4/9 = 0.44$ | $2.17$ | $4/2.17 = 1.85$ |

| Node | Shortest paths between others that pass through node | Betweenness |
|---|---|---|
| $A$ | 5 of 6; all except $D - E$ | $5/6 = 0.83$ |
| $D$ | 3 of 6; $A - E, B - E, C - E$ | $3/6 = 0.50$ |
| $E$ | 0 of 6 | $0$ |




Betweenness for $A$ does **not** count paths starting from $A$. It counts shortest paths between the other nodes $\{B, C, D, E\}$ that pass through $A$: $B - C$, $B - D$, $B - E$, $C - D$, and $C - E$ do; $D - E$ does not.

# Use of Betweenness Centrality — Vulnerability




The red broker lies on most shortest paths between clusters.
Remove it: the largest connected component shrinks sharply.

**Proposal.** Use betweenness as a stress-test tool.

1. Compute node or edge betweenness.
2. Remove the top-ranked nodes/edges.
3. Track largest connected component size.
4. Compare against random removal.

If targeted removal fragments the graph much faster than random removal, the network depends on a small set of brokers or bridges. For large graphs, approximate by sampling source nodes.

# Eigenvector Centrality — Recursive Prestige

**Node equation.**

$$C_E(v) = \frac{1}{\lambda} \sum_u A_{vu} C_E(u)$$

**Vector equation.**

$$Ax = \lambda x$$

**Meaning.** Important nodes are connected to important nodes.

**Theory.** $x$ is the leading eigenvector of $A$; this is recursive prestige (Bonacich 1987).

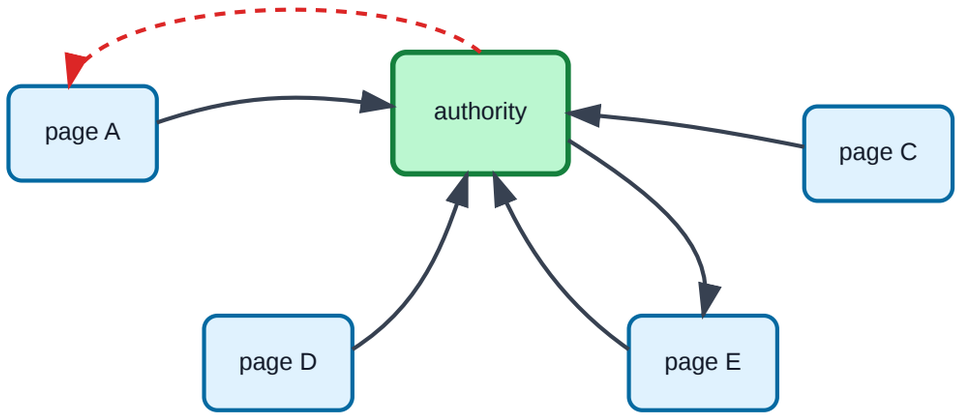
**Range.** Scores depend on normalization, often $0$ to $1$.

**Complexity.** Power iteration costs $O(k(n+m))$ for $k$ iterations, often written $O(km)$ on connected sparse graphs:

$$x^{(t+1)} = \frac{Ax^{(t)}}{\|Ax^{(t)}\|}$$

The two equations are the same idea: the score of each node equals the weighted prestige of its neighbors; in vector form this is the eigenvector equation.

# Use of Eigenvector Centrality — PageRank



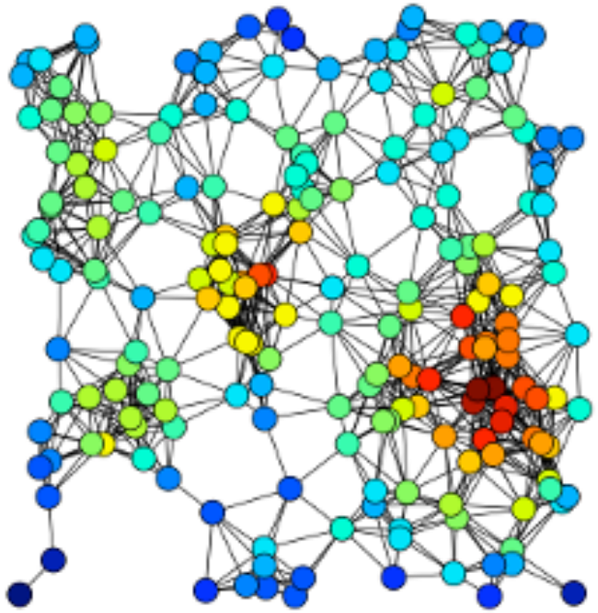

Follow links with probability alpha; randomly jump with probability 1-alpha.
Power iteration estimates the long-run visit probability of each page.

$$PR(v) = \frac{1-\alpha}{n} + \alpha \sum_{u \to v} \frac{PR(u)}{\text{outdeg}(u)}$$

**Random surfer.** With probability $\alpha$, follow a link. With probability $1-\alpha$, jump to a random page.
**Range.** PageRank scores are probabilities: each score is in $[0, 1]$ and all scores sum to $1$.
**Power iteration.** Repeatedly update all scores until they converge. Each iteration is sparse: $O(n+m)$, often written $O(m)$ on connected graphs.

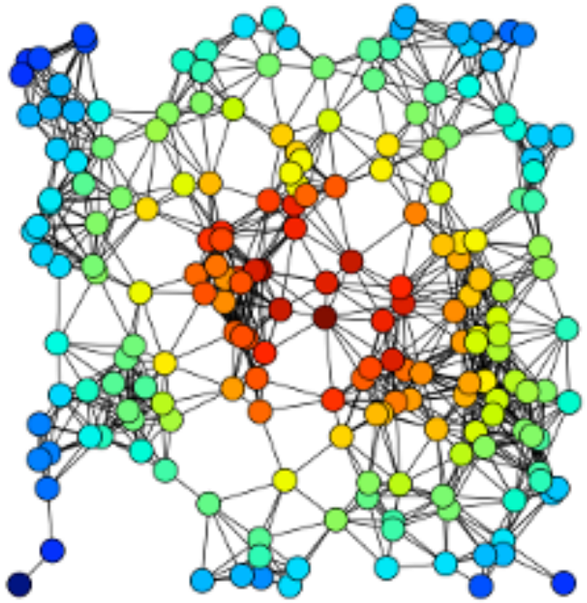
PageRank scales well by power iteration, but it is cheatable: link farms manufacture endorsements. Harmonic centrality is harder to game that way, but exact computation is more expensive.

# Same Graph, Four Views of Importance



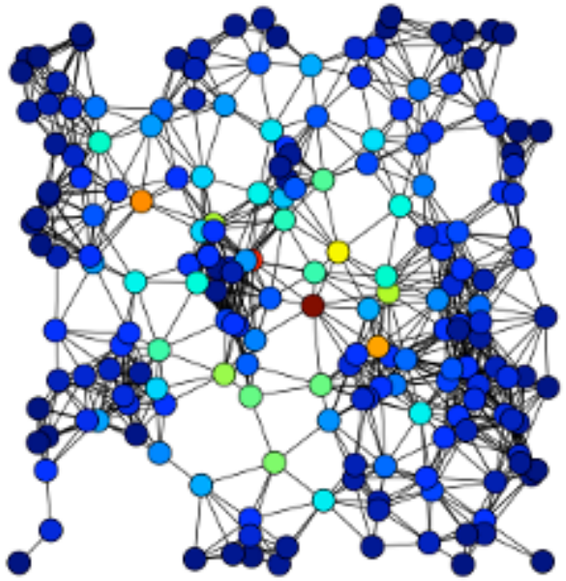
*Source: Easley & Kleinberg 2010*

**Degree**



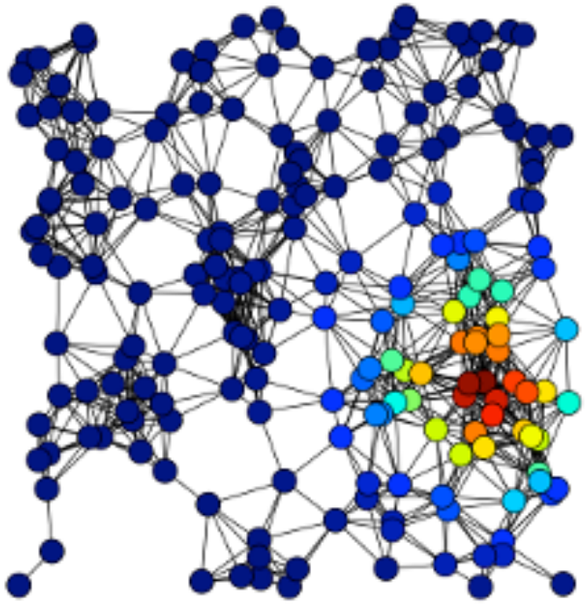
*Source: Easley & Kleinberg 2010*

**Closeness**



*Source: Easley & Kleinberg 2010*

**Betweenness**



*Source: Easley & Kleinberg 2010*

**Eigenvector**

Node size in each panel encodes that panel's centrality value. The same graph yields different "most important" nodes depending on which measure is used — confirming that centrality is always a theory of importance, not a neutral measurement.

# Centrality Summary

Notation: $n = |V|$ nodes, $m = |E|$ edges, $k =$ power-iteration steps. "One node" means scoring one focal node; "all nodes" means computing all scores. Sorting a complete ranking adds $O(n \log n)$.

| Measure | Theory | One node | All nodes |
|---|---|---|---|
| **Degree** | direct exposure | $O(\deg v)$, often $O(1)$ if stored | $O(n + m)$ |
| **Closeness** | short-path access | $O(n + m)$ BFS | $O(n(n + m))$ |
| **Harmonic** | reachable proximity | $O(n + m)$ BFS | $O(n(n + m))$ |
| **Betweenness** | path brokerage | $O(n(n + m))$ exact | $O(n(n + m))$ with Brandes |
| **Eigenvector / PageRank** | recursive prestige | computed as whole vector | $O(k(n + m))$ |

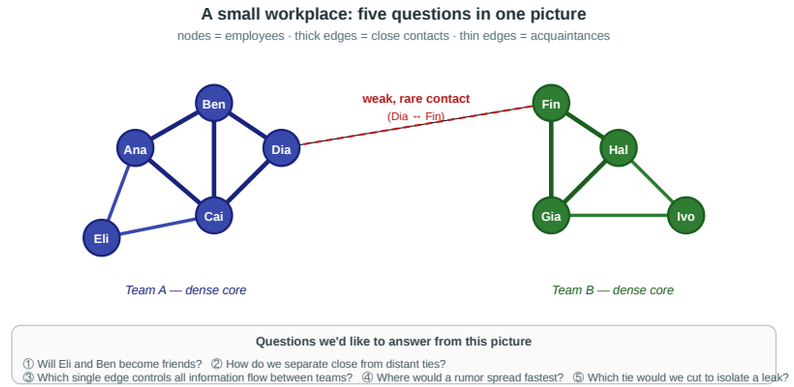
For closeness and harmonic centrality, moving from one focal node to all nodes multiplies the shortest-path computation by about $n$. For exact betweenness, however, even one focal node already requires all-source shortest-path dependency information, so Brandes computes all nodes for essentially the same asymptotic cost.

With positive weighted edges, replace BFS by Dijkstra: one source is roughly $O(m + n \log n)$ and all sources $O(nm + n^2 \log n)$. Weighted Brandes is commonly stated with the same $O(nm + n^2 \log n)$ distance-search term.

# Back to the Workplace

In the workplace figure, each measure picks a different winner:

- **Degree** → Ana (or Ben): most direct contacts inside team A's core
- **Closeness** → Dia: smallest average distance to everyone, because she sits on the only bridge
- **Betweenness** → Dia *and* Fin: every shortest path between the teams passes through both
- **Eigenvector** → Ana or Ben: connected to densely-connected others, which amplifies recursively



# Takeaway

> Centrality is always tied to a theory of influence, access, or control. Choosing a measure is choosing a theory — not collecting an objective fact.

# Quick Check

For each scenario, decide which centrality measure is most appropriate and justify briefly.

1. You want to identify the most influential gossip in a small village.
2. You want to find emergency-response hubs that can reach all districts within minimum time.
3. You want to detect which router, if attacked, would slow internet traffic between continents most.
4. You want to find the most prestigious researcher in a citation network.

# Quick Check

For each scenario, decide which centrality measure is most appropriate and justify briefly.

1. You want to identify the most influential gossip in a small village.
2. You want to find emergency-response hubs that can reach all districts within minimum time.
3. You want to detect which router, if attacked, would slow internet traffic between continents most.
4. You want to find the most prestigious researcher in a citation network.

**Solution:**

1. **Degree** — direct contacts matter.
2. **Closeness** — minimum average distance to districts.
3. **Betweenness** — router lies on intercontinental paths.
4. **Eigenvector / PageRank** — prestige flows from being cited by prestigious others.

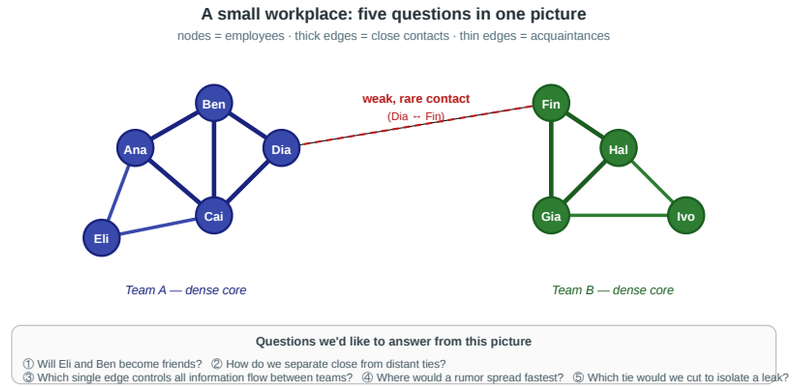
# What Counts as a Community?

**Guiding Question:** Can we give "community" a mathematical definition precise enough that an algorithm could find one?

# Working Definition

**Community (informal).** A subset of nodes $S \subseteq V$ with

- relatively many edges *inside* $S$
- relatively few edges *leaving* $S$
- density meaningfully higher than the surrounding graph.




Different formal definitions instantiate this intuition differently. The most influential is **modularity**, which compares observed internal density against a random null model.

# Modularity — The Canonical Objective

**Modularity** $Q$ of a partition $\{C_1, \ldots, C_k\}$ of a graph with $m$ edges and adjacency matrix $A$:

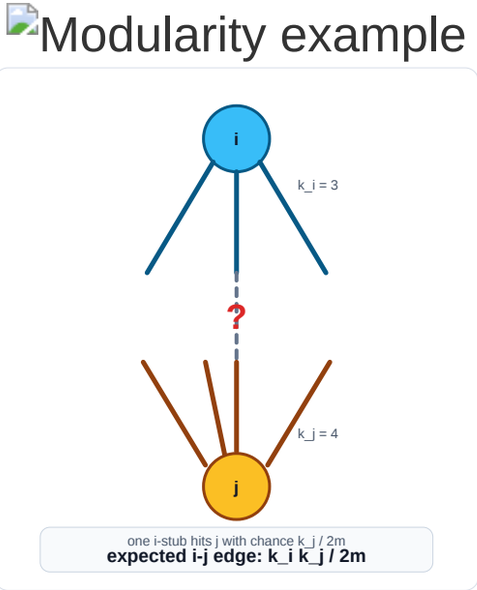
**Pairwise form**

$$Q = \frac{1}{2m} \sum_{i,j} \left( A_{ij} - \frac{k_i k_j}{2m} \right) \delta(c_i, c_j)$$

**Equivalent grouped form**

$$Q = \sum_{c} \left( \frac{l_c}{m} - \left( \frac{d_c}{2m} \right)^2 \right)$$

where $k_i$ is node degree, $\delta(c_i, c_j) = 1$ for same-community pairs, $l_c$ is the number of internal edges in community $c$, and $d_c$ is the sum of degrees in $c$.

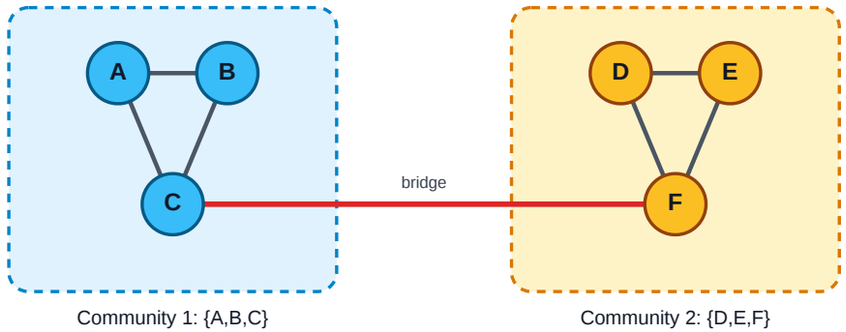



**Interpretation.** For every within-community pair, compare the *observed* edge $A_{ij}$ to the *expected* edge density $k_i k_j / 2m$ under a random rewiring that preserves degrees (the **configuration model**). $Q$ is the total surplus of within-community edges over chance.

$Q$ ranges roughly in $[-\frac{1}{2}, 1]$. Values in $[0.3, 0.7]$ typically indicate significant community structure; $Q \approx 0$ means no more internal density than expected by chance.

# Modularity — Worked Example

Modularity is calculated for the **full graph** together with a **given partition**. Here all six nodes are assigned to two communities; then $Q$ asks whether this complete partition has more internal edges than expected by chance.




m = 7 edges; each community has 3 internal edges and total degree 7

Given partition: $C_1 = \{A, B, C\}$ and $C_2 = \{D, E, F\}$. The bridge is the only between-community edge.

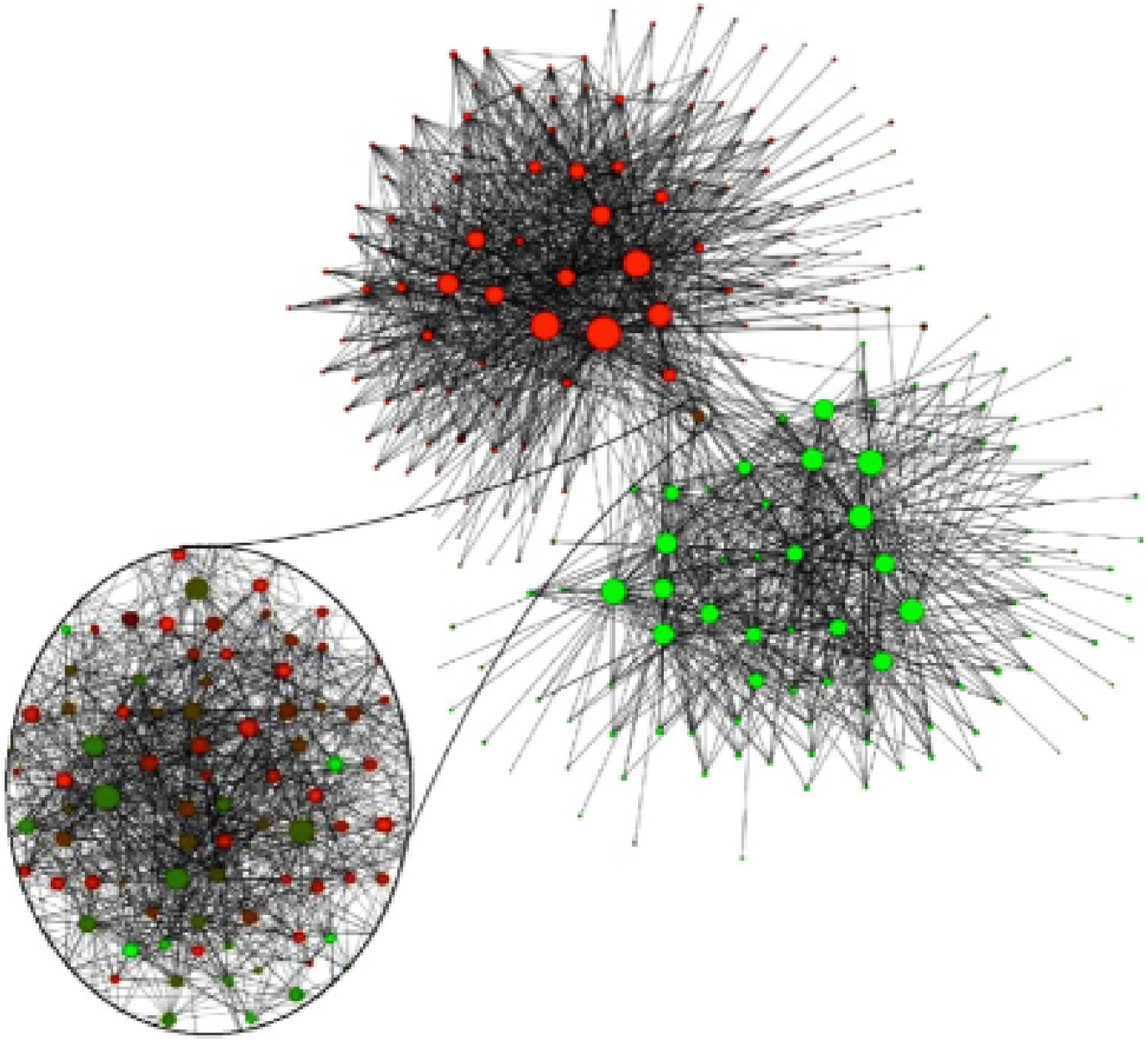
For each community:

- internal edges: $l_c = 3$
- total degree: $d_c = 2 + 2 + 3 = 7$
- full graph edges: $m = 7$

$$Q = 2\left(\frac{3}{7} - \left(\frac{7}{14}\right)^2\right) = 2(0.429 - 0.25) \approx 0.36$$

Interpretation: the partition has more within-community edges than expected under degree-preserving random rewiring, so $Q$ is positive.

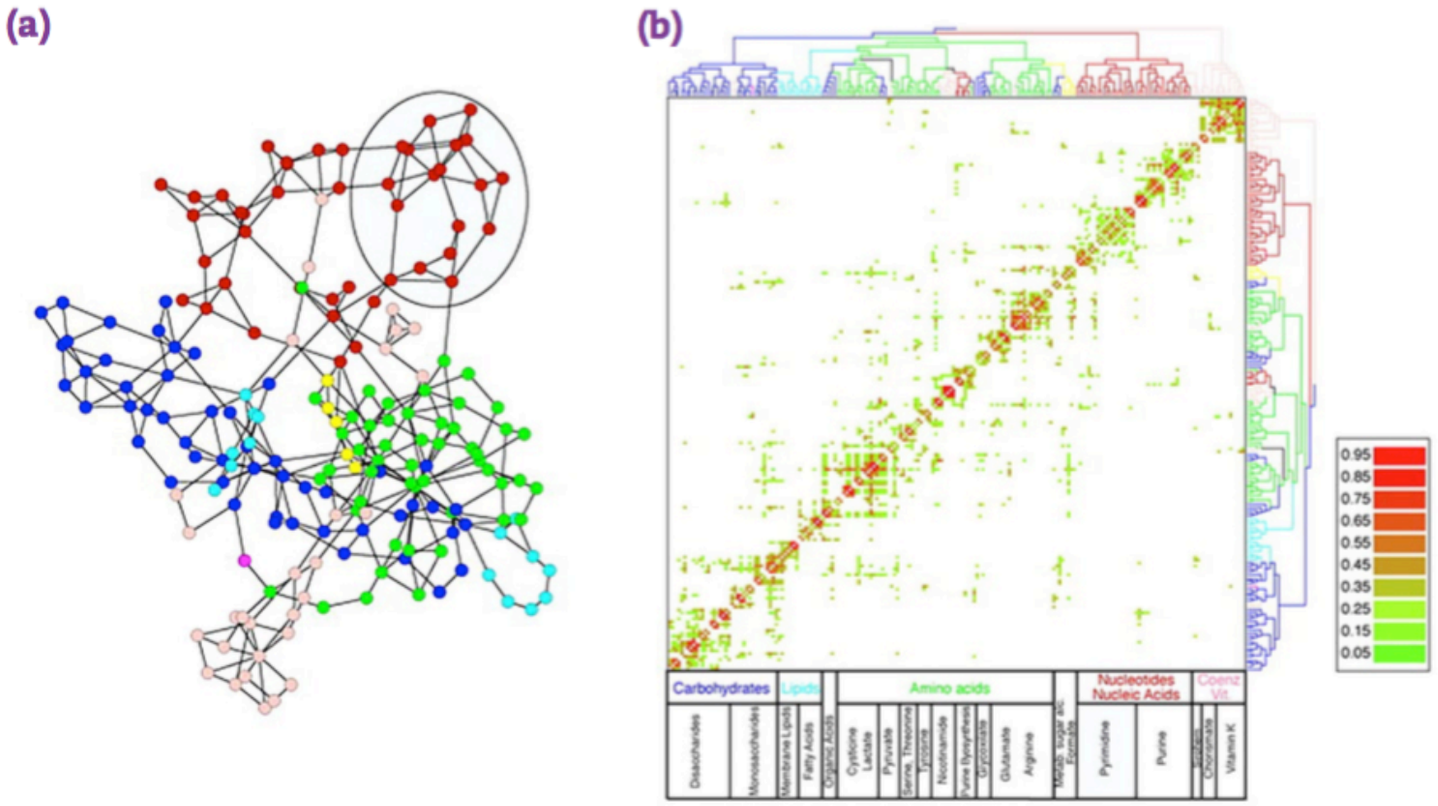
# Example Case — Belgian Phone Calls



Nodes are phone users or regions; edges are call relationships. Community detection recovers dense calling regions, and the strongest split aligns with Belgium's Flemish-Walloon linguistic divide — a social boundary inferred from communication structure, not supplied as a label.

*Source: Easley & Kleinberg 2010*

# Example Case — Metabolic Networks




*Source: Easley & Kleinberg 2010*

Nodes are metabolites or reactions; edges represent biochemical transformations. Communities correspond to functional modules: groups of reactions that interact densely because they participate in the same pathway or cellular process.

# The Catch — Modularity Maximization Is NP-Hard

> **Result (Brandes et al., 2008).** Finding the partition that maximizes $Q$ on a general graph is **NP-hard**. No polynomial-time algorithm for the exact optimum is known.

# The Catch — Modularity Maximization Is NP-Hard

> **Result (Brandes et al., 2008).** Finding the partition that maximizes $Q$ on a general graph is **NP-hard**. No polynomial-time algorithm for the exact optimum is known.

Sound familiar? This is the exact same situation as **MaxSTC** in Lecture 03: a theoretically clean objective that is computationally out of reach on real data.

# The Catch — Modularity Maximization Is NP-Hard

> **Result (Brandes et al., 2008).** Finding the partition that maximizes $Q$ on a general graph is **NP-hard**. No polynomial-time algorithm for the exact optimum is known.

Sound familiar? This is the exact same situation as **MaxSTC** in Lecture 03: a theoretically clean objective that is computationally out of reach on real data.

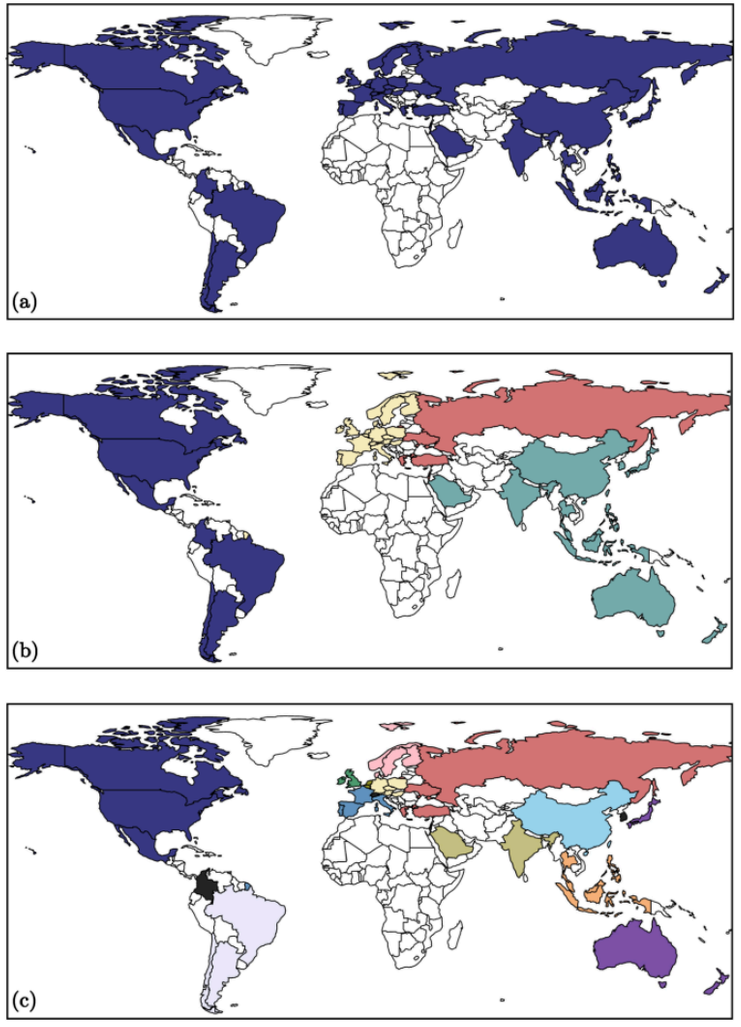
**Consequence.** The rest of the module introduces two families of polynomial-time *heuristics* that find partitions with high $Q$ without guaranteeing the global maximum:

- **divisive** (Girvan–Newman): peel apart high-betweenness edges
- **agglomerative** (Louvain, Leiden): greedily merge nodes that increase $Q$

# Study — International Trade Communities

Cho, Lee & Kim (2023) model international trade as a network of countries connected by trade relationships and detect **trade communities at multiple resolutions** (Cho et al. 2023).

**Network formation.** Nodes are countries; weighted edges encode trade relationships. Strong edges form when countries exchange large volumes or are tightly embedded in the same trade system.
**Question.** Does world trade split into one stable map of blocs, or into nested communities that depend on resolution?
**Finding.** A single community scale can hide structure: large blocs contain smaller regional or sectoral sub-blocs. Economic communities are not only geographic; they also reflect trade intensity, specialization, agreements, and supply-chain structure.




*Source: Cho et al. 2023 — CC BY 4.0*

**Panels**
$\Phi^*$ is the **cooccurrence threshold**: countries are grouped only if they appear together in the same community in at least that share of multiresolution runs. Higher $\Phi^*$ keeps only more stable links.

- **a** $\Phi^* = 0$: one large trade component
- **b** $\Phi^* = 0.41$: four broad blocs
- **c** $\Phi^* = 0.96$: 16 stable country groups

Figure: selected country configurations at cooccurrence thresholds $\Phi^* = 0$, $0.41$, and $0.96$; reproduced from Fig. 4 in Cho, Lee & Kim (2023), CC BY 4.0.

# Takeaway

Modularity $Q$ is the canonical mathematical definition of "community": excess internal density over a random-graph null model. Maximizing it exactly is NP-hard — so every practical community detection method is a heuristic.

# Quick Check

A graph has two dense clusters of 50 nodes each connected by a single edge, plus one small triangle attached to one cluster via a single edge.

1. Roughly how many natural communities does this graph have?
2. Will modularity maximization find all of them?
3. If you asked a balanced 2-partition algorithm to split it into two equal halves, what might go wrong?

# Quick Check

A graph has two dense clusters of 50 nodes each connected by a single edge, plus one small triangle attached to one cluster via a single edge.

1. Roughly how many natural communities does this graph have?
2. Will modularity maximization find all of them?
3. If you asked a balanced 2-partition algorithm to split it into two equal halves, what might go wrong?

**Solution:**

1. **Three** natural communities: two dense clusters plus the triangle.
2. On this small graph, likely yes; at larger scale, modularity's **resolution limit** can hide small groups.
3. Balanced 2-partition may cut through a dense cluster or merge the triangle with the wrong side.

# Finding Communities — Divisive and Agglomerative

**Guiding Question:** If modularity maximization is NP-hard, how do we find good partitions in practice?

# Two Strategies

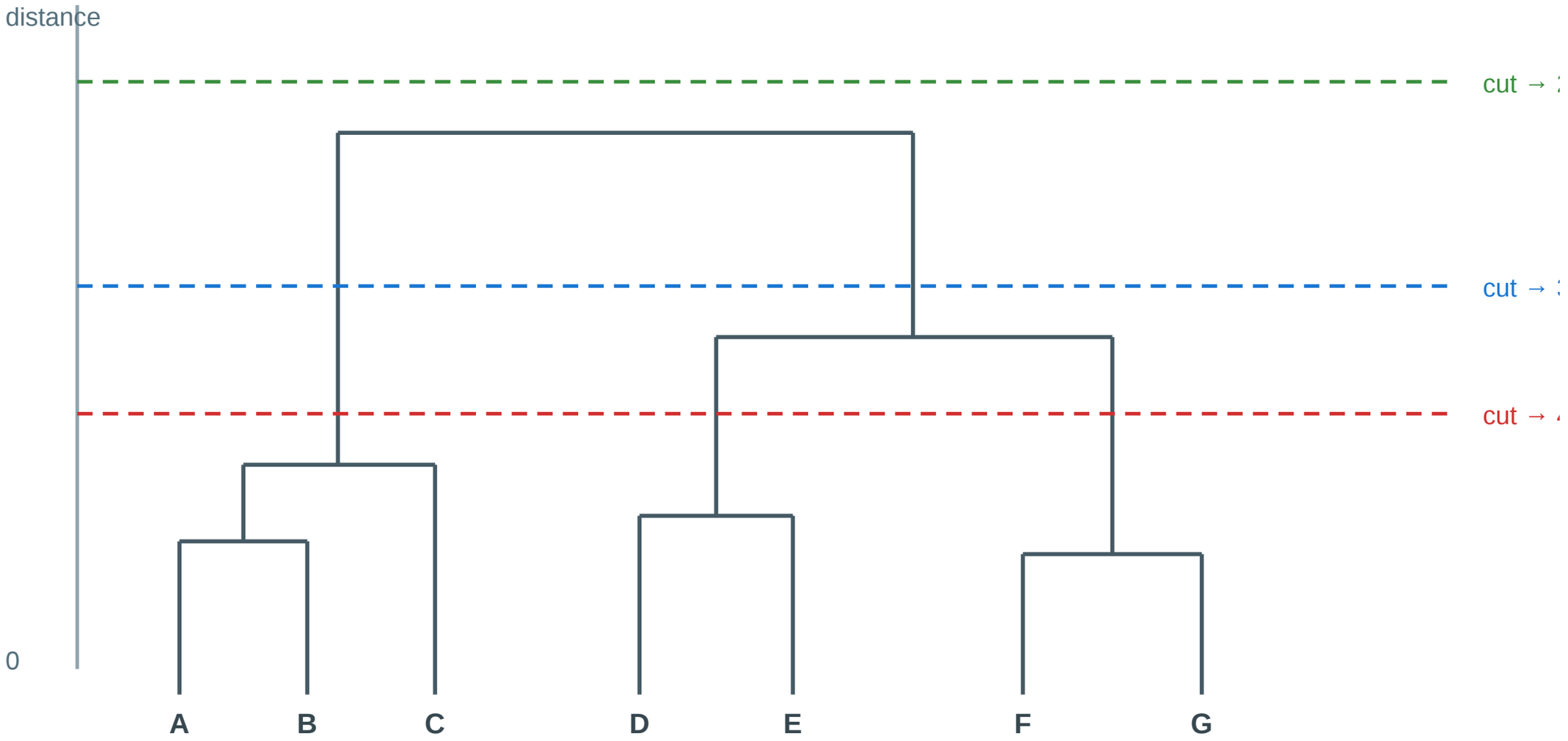
**Divisive (top-down).** Start with the whole graph as one cluster. Repeatedly remove edges that lie *between* communities, peeling the graph apart. Example: Girvan–Newman.

**Agglomerative (bottom-up).** Start with each node as its own cluster. Repeatedly merge the pair whose merge most *increases* modularity. Example: Louvain, Leiden.

Both strategies produce a hierarchy — a **dendrogram** — rather than a single flat partition. Where to cut the dendrogram is the analyst's modeling decision.

# The Dendrogram as Output

# Hierarchical clustering: the cutoff is a modeling choice




*Each horizontal cut yields a different clustering — the analyst chooses the level*

# Girvan–Newman — The Divisive Algorithm

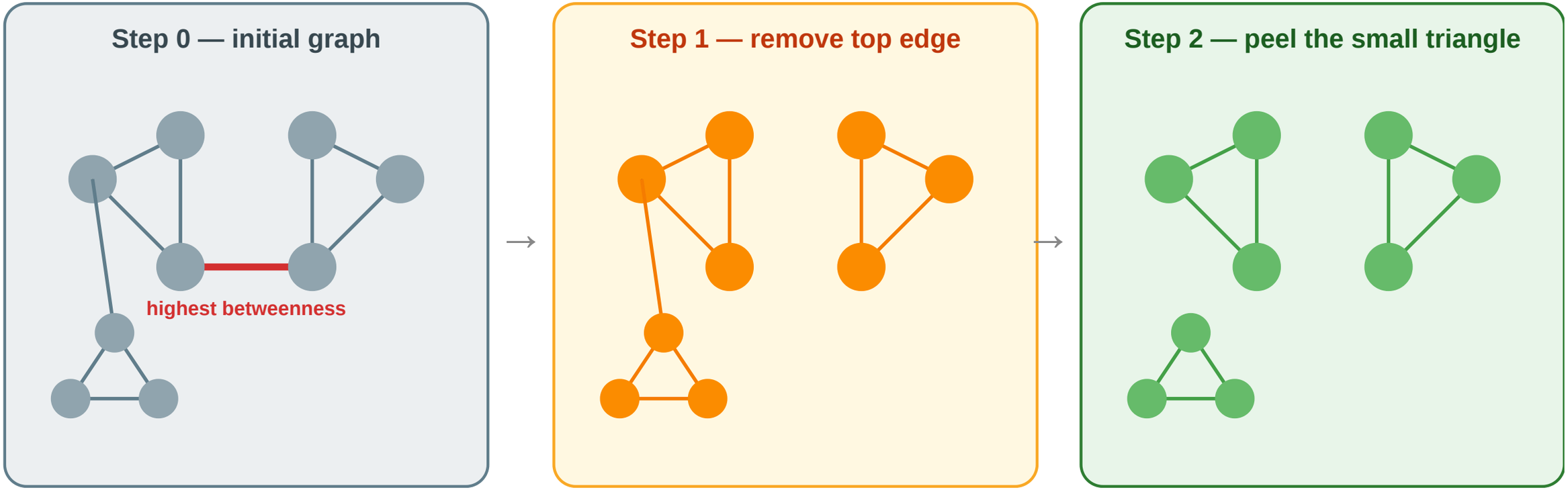
**Algorithm: Girvan–Newman**

*(Input: Graph $G = (V, E)$)*

1. Compute the **edge betweenness** of every edge — the fraction of all shortest paths that pass through it.
2. Remove the edge with the **highest** edge betweenness.
3. Recompute edge betweenness on the remaining graph.
4. Repeat until no edges remain — record the partition at each step.
5. Choose the partition that maximizes **modularity** $Q$.

Edges with high edge-betweenness are exactly the **local bridges** and **weak ties** from Lecture 03 — they carry many shortest paths because all cross-community communication must cross them. Girvan–Newman is an algorithmic version of Granovetter's weak-tie argument.

# Girvan–Newman — Worked Example



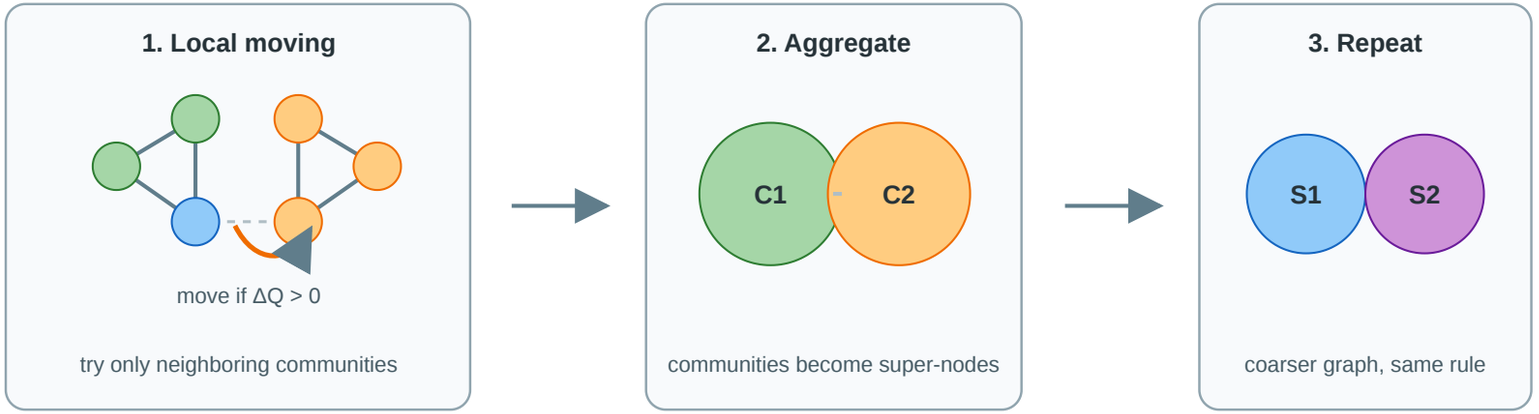

*two main clusters revealed*

- **Step 0:** the bridge edge has highest betweenness because every path between the two main clusters must cross it.
- **Step 1:** removing that bridge reveals two main clusters; the small triangle remains weakly attached to the left cluster.
- **Step 2:** the next high-betweenness edge is the link to the triangle; removing it isolates the third community.
- **Final partition:** three communities.

# Louvain — Greedy Modularity in Two Phases

**Louvain (Blondel et al., 2008)** is a fast greedy heuristic for high-modularity partitions.

**Louvain: local moves + aggregation, repeated until Q stops improving**




**No pre-given k:** the number of communities emerges from accepted modularity-improving moves and later aggregation levels.

For a node v, Louvain tests the current communities found among v's neighbors, plus staying where it is.

## Algorithm Sketch

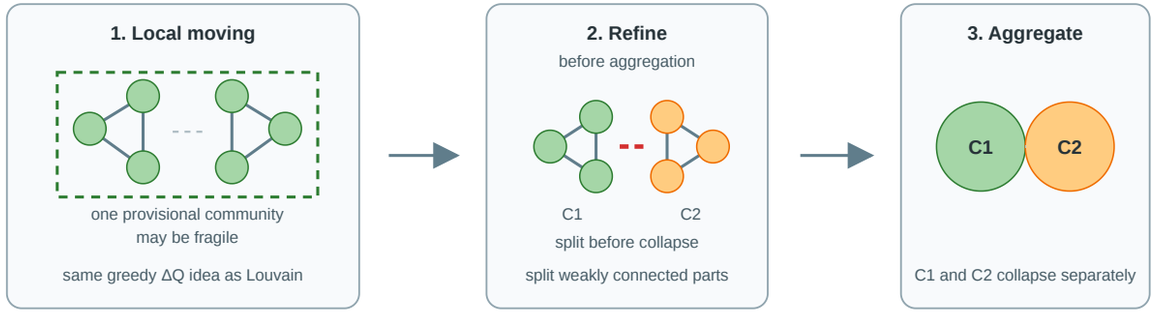
- **Initialization:** start with $k = n$ communities: every node is its own community.
- **Local moving:** visit node $v$ and move it to another community only if this increases $Q$.
- **Candidate communities:** for $v$, test the current communities of $v$'s neighbors, plus staying where it is. At the first pass these candidates are just neighboring single-node communities.
- **Aggregation:** collapse each detected community into a super-node.
- **Repeat:** run local moving again on the smaller graph until modularity no longer improves.
- **No fixed $k$:** the number of communities is not pre-given; it emerges from the greedy moves and aggregation levels.

**Drawbacks:** Louvain is greedy, so results can depend on node order and local optima. It inherits modularity's resolution limit and can produce internally weak or disconnected communities because it aggregates provisional groups without a

# Leiden — Louvain with Refinement

**Leiden (Traag et al., 2019)** keeps Louvain's speed idea but adds a refinement phase before aggregation.

**Leiden: Louvain-style moves plus refinement before aggregation**




**Refinement check:** a community found by local moving is not automatically collapsed into one super-node.

Leiden first splits weakly connected provisional communities, then aggregates only the refined pieces.

## Algorithm Sketch

- **Local moving:** first improve $Q$ with Louvain-style node moves.
- **Refinement:** inspect each provisional community before aggregation. Start again with its nodes as small subcommunities, then merge/move only if the subcommunity remains well connected and improves the quality function.
- **Split effect:** if a Louvain community is held together only by a weak bridge, Leiden can keep its parts separate instead of collapsing them into one super-node.
- **Aggregation:** collapse only the refined communities and repeat on the smaller graph.
- **No fixed $k$:** Leiden also lets $k$ emerge, but it prevents weak provisional groups from being locked in too early.

Both, Louvain and Leiden, are heuristic modularity optimizers: they do not guarantee the global maximum, but they scale to graphs where Girvan-Newman is infeasible. Leiden is usually preferred in research pipelines because it preserves

# Graph Partitioning — Cut-Based Methods

**Min-cut / Max-flow (Ford–Fulkerson)** Find the smallest set of edges whose removal disconnects two specified groups. Exact in polynomial time — but minimizes raw cut size, not normalized density.

**Kernighan–Lin (1970)** Local search: initialize a balanced 2-partition at random; iteratively swap node pairs to decrease the cut size. Fast in practice; converges to a local optimum.

**Spectral partitioning** Compute the Laplacian $L = D - A$ of the graph. The **Fiedler vector** (eigenvector of the second-smallest eigenvalue $\lambda_2$ ) encodes the natural 2-partition: nodes with positive entries go left, negative entries go right.

$\lambda_2$ is the **algebraic connectivity**: $\lambda_2 = 0$ means the graph is already disconnected. Spectral methods generalize to $k$ communities by using the $k$ smallest eigenvectors — the basis of **spectral clustering**.

# Embedding-Based Community Detection

## Node embeddings + clustering

**node2vec (Grover & Leskovec, 2016)** Random-walk–based embedding: simulate biased random walks on the graph, then train Word2Vec–style skip-gram to produce a low-dimensional vector per node. Communities emerge as clusters in embedding space.

**Pipeline:** node2vec → $k$-means / DBSCAN → community labels

Hyperparameter $p$ controls return probability (DFS-like, captures communities), $q$ controls exploration (BFS-like, captures roles). Setting $p \gg q$ favors dense cluster detection.

## Graph Neural Networks

**GNN-based community detection** Train a GNN encoder (e.g. GCN, GraphSAGE) to produce embeddings that maximize an unsupervised objective — modularity, contrastive loss, or cluster assignment entropy. The decoder maps embeddings to community memberships.

| Approach | Key idea |
|---|---|
| **GRAPH-BERT** | Self-supervised pre-training on subgraphs |
| **DMGI** | Contrastive multiplex embeddings |
| **DMoN** | Differentiable modularity as loss |

GNNs naturally handle **overlapping communities** (soft cluster assignment) and **attributed graphs** (node features + topology jointly).

# Algorithm Comparison

| Method | Strategy | Complexity | Strength | Limitation |
|---|---|---|---|---|
| **Min-cut** | Cut minimization | e.g. $O(nm)$ for basic max-flow variants | Exact for 2-partition | Ignores density, favors small cut |
| **Kernighan–Lin** | Local swap | $O(n^2)$ per pass | Fast for balanced partition | Local optimum, fixed $k$ |
| **Spectral** | Laplacian eigenvectors | $O(n^3)$ full / faster sparse approx | Global structure, principled | Slow exact; needs $k$ in advance |
| **Girvan–Newman** | Edge betweenness | $O(nm^2)$ worst case | Hierarchy, no $k$ needed | Too slow for large graphs |
| **Louvain / Leiden** | Modularity greedy | near-linear in $m$ empirically | Fast, scalable, no $k$ needed | Resolution limit, local opt |
| **node2vec + clustering** | Random-walk embed | walks $O(rnl)$ + training/clustering | Scales, flexible | Needs $k$, walk hyperparams |
| **GNN-based** | Neural message passing | often $O(TLmd)$-like | Soft membership, features | Requires training objective |

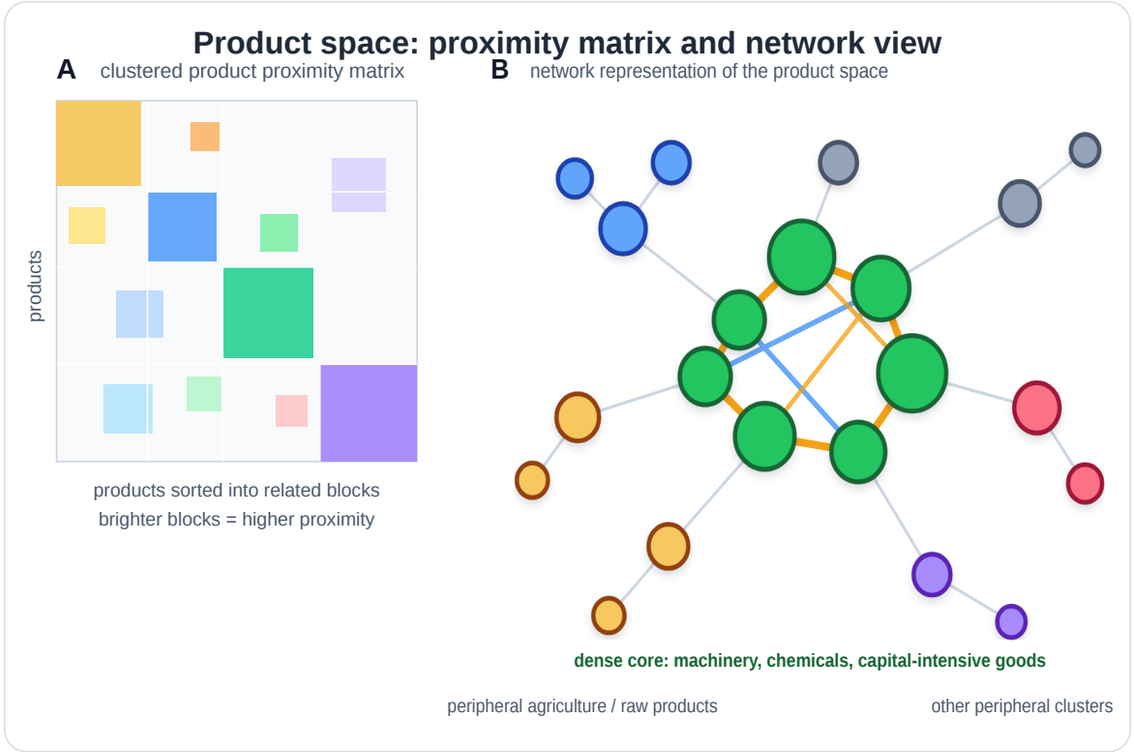
# Study — Product Space and Development

> Hidalgo, Klinger, Barabasi & Hausmann (2007) build a **product-space network**: nodes are exported products, and edges connect products that require similar capabilities (Hidalgo et al. 2007).

**Network formation.** Nodes are SITC-4 product classes. A country is linked to a product when it exports it with **revealed comparative advantage** (exports more than expected relative to the world average, $RCA > 1$). Two products are similar if countries that export one competitively also tend to export the other competitively:

$$\phi_{ij} = \min\{P(RCA_i > 1 \mid RCA_j > 1), P(RCA_j > 1 \mid RCA_i > 1)\}.$$

Edges therefore do **not** mean product-to-product trade. They reveal shared capabilities, knowledge, institutions, inputs, or infrastructure.




> Redrawn teaching schematic after Fig. 1 in Hidalgo et al. (2007), DOI: 10.1126/science.1144581.

**Question.** Why can some countries diversify into sophisticated products while others remain stuck in low-complexity export baskets?

**Finding.** Sophisticated products sit in a dense core; less sophisticated products often lie in sparse peripheral regions. Countries diversify by moving to nearby products, so peripheral position limits feasible development paths.

# Takeaway

Divisive and agglomerative heuristics approximate the NP-hard modularity optimum in polynomial time. Girvan–Newman is the didactic divisive method; Louvain and Leiden are the scalable agglomerative methods used in practice. Graph partitioning (min-cut, spectral) targets fixed-$k$ balanced splits; embedding-based methods (node2vec, GNNs) detect communities from representation space and can handle overlapping memberships and node features.

# Quick Check

You run Girvan–Newman on a small graph and get a dendrogram with a maximum $Q$ at the cut that produces 4 communities. You then run Louvain on the same graph and it reports 3 communities with a slightly lower $Q$.

1. Which answer is "correct"?
2. What could explain the discrepancy?

# Quick Check

You run Girvan–Newman on a small graph and get a dendrogram with a maximum $Q$ at the cut that produces 4 communities. You then run Louvain on the same graph and it reports 3 communities with a slightly lower $Q$.

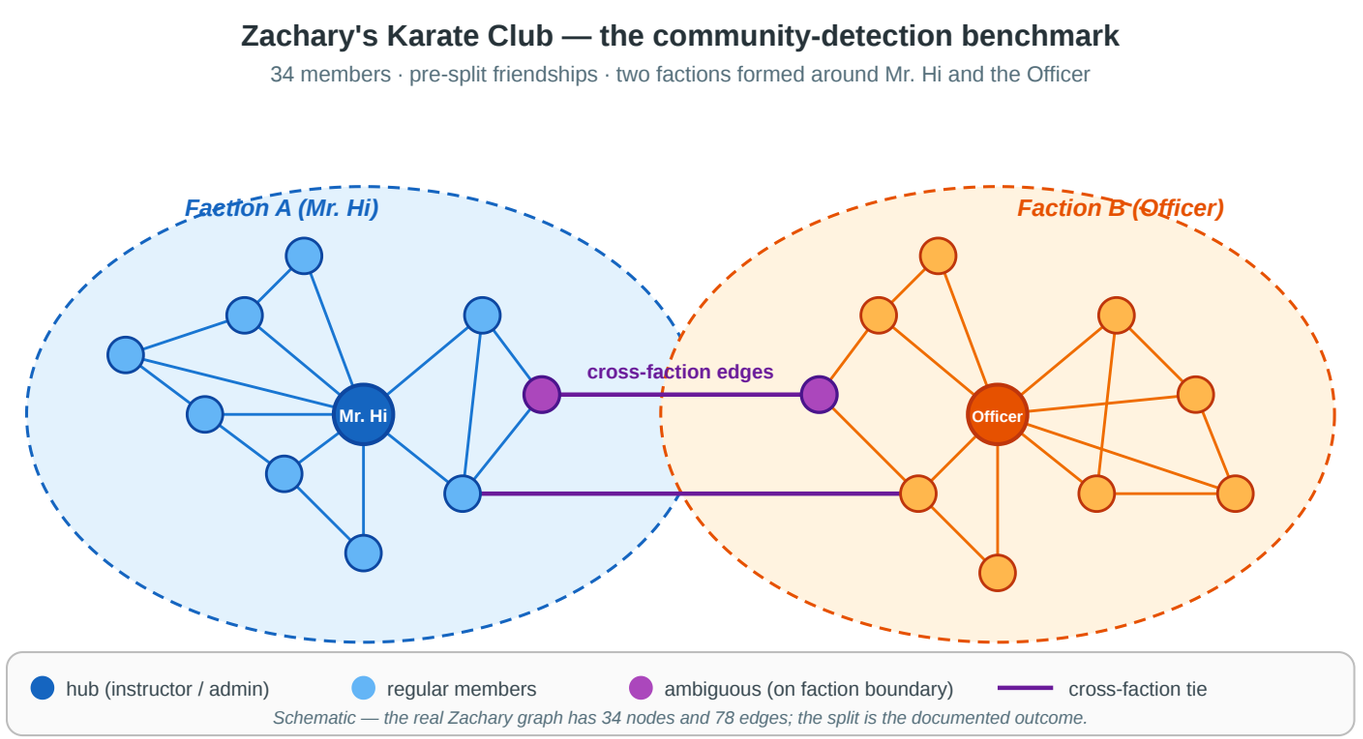
1. Which answer is "correct"?
2. What could explain the discrepancy?

> **Solution:** Neither is absolutely correct. Both are heuristics approximating an NP-hard objective. Louvain may get stuck in a local optimum; Girvan–Newman may expose finer but noisy structure; modularity has a resolution limit; the graph may have multiple near-optimal partitions.

# Empirical Anchor — Zachary's Karate Club

**Guiding Question:** Do community-detection heuristics actually recover real social groups — and how do we know?

# The Story




**Reference.** Zachary (1977), *An Information Flow Model for Conflict and Fission in Small Groups*, **J. Anthropological Research 33, 452–473**.

Wayne Zachary observed a university karate club for two years. A conflict arose between the instructor (**Mr. Hi**) and the chief administrator (**Officer**), and the club eventually **split in two** — with each member following one or the other.

# What the Algorithms Find

| Method | Typical result |
|---|---|
| **Girvan–Newman** | Recovers the two factions with **1 misclassified** node |
| **Louvain / Leiden** | Recovers the two factions; typically finds **4 sub-groups** at max $Q$ |
| **Max-modularity (exact on this small graph)** | Finds the same 4 sub-groups nested inside the 2 factions |

> The ~1-node misclassification is structurally meaningful: the node in question genuinely sat on the boundary between the two factions during the conflict period, and which side they joined was a close call even to Zachary himself.

# What the Study Could Not See

Even in this iconic success case, the empirical validation has blind spots that parallel Lecture 03's gap between construct and measurement:

**1. Modularity resolution limit (Fortunato & Barthélemy 2007).** On the 34-node karate graph the exact max-$Q$ partition has **4** communities, not **2**. The ground-truth split is not even the modularity optimum — the algorithm is answering "which partition maximizes $Q$", not "which partition matches the real factions".

**2. Binary ground truth.** Zachary's data records which faction each member *eventually* joined — but not the *strength* of their preference, their uncertainty during the conflict, or whether a third "neutral" group briefly existed. The true social structure may have been continuous or overlapping.

**3. Static snapshot.** The friendships were recorded at a single time. The dynamics of the conflict — who influenced whom, when someone shifted — are invisible to the algorithm.

# Takeaway

Zachary's karate club shows that community-detection heuristics recover real social structure from graph topology alone — but also that even the "canonical" answer has blind spots: modularity's resolution limit, binary ground truth, and static snapshots all hide real structure the algorithm cannot see.

# Wrap-Up — Closing the Modeling Loop

Across Lectures 03 and 04 we have traversed the full modeling/measurement loop twice:

| Level | Theoretical ideal | NP-hard recovery | Polynomial proxy | Empirical test |
|---|---|---|---|---|
| **Edges (L03)** | Strong/weak labeling | MaxSTC | Clustering coeff., overlap | Kossinets–Watts email |
| **Nodes / Groups (L04)** | Modularity-optimal partition | Modularity maximization | Girvan–Newman, Louvain, Leiden | Zachary karate club |

Every concept in the two lectures occupies one of those four cells.

## Reading

Chapter 3.6 and Chapter 9 of Easley & Kleinberg (2010) cover community structure, brokerage, and centrality. Newman (2010), Chapter 7, gives the formal treatment of centrality measures. For modularity and Louvain, see Blondel et al. (2008) and Traag et al. (2019) for Leiden.

# Image Credits & References


Barab'asi, Albert-L'aszl'o and Albert, R'eka (1999). Emergence of Scaling in Random Networks. *Science*.

Bonacich, Phillip (1987). Power and Centrality: A Family of Measures. *American Journal of Sociology*.

Burt, Ronald S. (1992). Structural Holes: The Social Structure of Competition. *Harvard University Press*.

Cho, Wonguk and Lee, Daekyung and Kim, Beom Jun (2023). A Multiresolution Framework for the Analysis of Community Structure in International Trade Networks. *Scientific Reports*.

Easley, David and Kleinberg, Jon (2010). Networks, Crowds, and Markets: Reasoning about a Highly Connected World. *Cambridge University Press*. https://www.cs.cornell.edu/home/kleinber/networks-book/

Hidalgo, C'esar A. and Klinger, Bailey and Barab'asi, Albert-L'aszl'o and Hausmann, Ricardo (2007). The Product Space Conditions the Development of Nations. *Science*.